

Caterpillar Asset Management V2 API

Meeting handout — what it is, how it works, and where we can use it

Reference: OpenAPI v2.1.0 spec + Asset Management V2 Developer Guide (SV-0066). Validated live against the production endpoint. Prepared June 2026.

At a glance

The Asset Management V2 API is the **authoritative system of record for who owns which piece of equipment** across the Caterpillar dealer network. It is small — **four operations** — but it sits on a rich ownership model and **enforces Caterpillar’s ownership-conflict policy automatically**.

What it does	Read and manage asset ownership records and ownership-transfer requests
Size	4 endpoints (1 read, 3 write)
Auth	OAuth2 client-credentials (Microsoft Entra ID); 60-minute tokens
Format	JSON over HTTPS
Environment	Production only — no sandbox (an isolated test dealer code is used instead)
Status with us	Validated end-to-end; 3 integrations prototyped (Copilot, Excel, Snowflake)

The one-sentence version: *it’s the system of record for who owns which asset — and it both answers that question and safely changes the answer.*

The vocabulary (everything below speaks in these terms)

Term	Meaning
Asset	A physical machine. Identified by make code + serial number .
Make code	Manufacturer code (e.g. <code>CW1</code>); dealers may use a 2-char dealer make code .

Term	Meaning
CCID	Caterpillar Customer ID — the end customer.
DCN	Dealer Customer Number — a specific dealer↔customer relationship.
Dealer code	The dealer organization (passed as partyNumber).
Ownership record	One asset↔customer association, keyed by make + serial + DCN + dealer .
Ownership type	owned , rental , leased , sold , inventory , unknown .
Relation status	ACTIVE (current) or PENDING (a transfer is in progress).

Anchor fact: one asset can have **many** ownership records at once — different dealers, DCNs, and types. The rule that keeps it coherent: **only one customer can be the ACTIVE owner at a time**. Everything below exists to enforce that.

The four endpoints

All four are `POST`. The asset identifier travels as query parameters; the payload travels in the JSON body.

Operation	Path	Reads/Writes	Success	Purpose
Search	<code>POST /ownershipRecords/search</code>	Read	<code>200</code> + records	Find who owns an asset
Add / Update	<code>POST /ownershipRecords</code>	Write	<code>200</code> (ACTIVE/PENDING)	Create, update, or re-activate a record
Expire	<code>POST /ownershipRecords/expire</code>	Write	<code>204</code>	Remove a record (relinquish ownership)

Operation	Path	Reads/Writes	Success	Purpose
Transfer	POST /ownershipRequests/transfer	Write	204	Approve / reject a pending transfer

Search — 1–2 exact-match filters on `dcn`, `serialNumber`, `assetName`, or `makeCode` (two combine with AND). Returns asset metadata, customer, and dealer association. **It is network-wide** — a serial search returns records across *all* dealers, not just yours.

Add / Update — if the record exists, only the fields you send change; if it's new, `ownershipTypeCode`, `model`, and `modelYear` are required. Setting an asset to `owned` triggers conflict handling (next page).

Expire — ends an ACTIVE or PENDING record. Expiring a PENDING record also cancels its transfer request.

Transfer — only the **current owner** can act. Approve → ownership moves; reject → it stays (a reason is required to reject).

Adding a record: what it actually takes

To **create a new** ownership record (`POST /ownershipRecords`), you supply an asset identity (query parameters) plus a small body. Three body fields are required *only because the record is new* — updates send just what changes.

Minimum to add a new record

Identity (query parameters): `partyNumber` (dealer code) · `serialNumber` · `makeCode` or `dealerMakeCode` · `dcn`

Body (required for a new record): `ownershipTypeCode` · `model` · `modelYear`

Optional body: `productFamilyCode`, `productFamilyName`, `baseAssetName`, `customAssetName`

Preconditions that must already be true (or the call is rejected):

- The **DCN already exists**, is **active**, and is tied to a valid customer (CCID). *This API attaches an asset to an existing dealer–customer relationship; it does not create the customer or the DCN.* Those are set up in the Customer Admin Tool first.
- The **make code is a recognized manufacturer** (see below).
- The **serial number matches the required format** (format depends on the make — see below).
- Your **credentials are entitled** to that dealer code.

Minimal example:

```
POST /ownershipRecords?partyNumber=ZZIO&makeCode=CAT&serialNumber=SERIAL1&dcn=DCN1
{ "ownershipTypeCode": "owned", "model": "980H", "modelYear": "2006" }
```

Make codes — a field we must get right

Every asset is identified by **make code + serial number**, so a make code is **mandatory on every add** — there is no way to create a record without one. There are two forms, and you provide **exactly one**:

Field	Format	What it is	Examples
<code>makeCode</code>	3 characters	Caterpillar's master manufacturer code	<code>CW1</code> (CAT), <code>KDC</code> (Komatsu), <code>FA1</code> (Ford), <code>G02</code> (Galion), <code>SB6</code> (Snow Wolf)
<code>dealerMakeCode</code>	2 characters	A dealer's own shorthand for a make	<code>CW</code> , <code>AA</code>

Things to know — and to plan for:

- **It must be a *valid, recognized code***, or the API rejects the call (`400.202` / `400.209` for `makeCode`, `400.208` for `dealerMakeCode`). You can't invent one.
- **CAT *and* non-CAT (competitive) makes are tracked**, each with its own code. So you must use the correct code for the *actual* manufacturer of the machine — a Komatsu asset needs the Komatsu make code, not a generic CAT one.
- **The make also drives serial-number validation**. CAT makes enforce the strict 8-character serial format (3 alphanumeric + 5 numeric); non-CAT makes allow the looser 4–50 character format. Pick the wrong make and a valid serial can be rejected.
- **Dependency to flag**: to populate this reliably at scale we need a **source of truth for valid make codes and our dealer make-code mappings**. This API doesn't hand back a make-code list — it validates against Caterpillar's reference data. Lining up that lookup is a prerequisite for any bulk add/onboarding work.

The important part: it encodes business rules, not just data

Most APIs are storage — your code keeps the data sane. This one **enforces an invariant for you**:

An asset can have only one owner at a time.

You never check that yourself. When a write would violate it, the API doesn't just reject — it **resolves the conflict per Caterpillar policy**.

The nuance most people miss: "owned" is tied to the *customer*, not the dealer

The exclusive claim isn't "a dealer owns it" — it's "a **customer (CCID)** owns it." That produces two facts that feel contradictory until you see the rule:

- **Multiple dealers can each hold an "owned" record for the same asset at once** — *as long as they point to the same customer.*
- **A conflict only arises when a *different* customer is made the owner.**

And only `owned` + `ACTIVE` triggers the check. `leased`, `rental`, `inventory`, and `PENDING` records never compete for the single "owner" slot — so this is all legal simultaneously:

```
Asset A1 → Customer 1 → owned    ← the one exclusive claim
Asset A1 → Customer 2 → leased    ✓ allowed
Asset A1 → Customer 3 → rental    ✓ allowed
```

What happens when you *do* hit a conflict — three outcomes

When you add/update a record to `owned` + `ACTIVE`, the API picks one path:

1. **No conflict** → the record becomes `ACTIVE` immediately.
2. **Conflict within your own dealership** → the API **silently expires** the conflicting record and activates yours. (You can reorganize your own customers.)
3. **Conflict with a different dealer** → the API **refuses to seize ownership**. It places your record in `PENDING`, **automatically creates an ownership transfer request** to the current owner, and marks your side `SENT` / their side `RECEIVED`. Your record only goes `ACTIVE` if they **approve**.

So the four endpoints are really one workflow

- ```
1. search → find the current owner
2. add "owned" → stake a claim
 └─ same dealer: auto-expire old → you are ACTIVE (instant)
 └─ different dealer: a PENDING transfer request is created
3. transfer → the current owner approves or rejects
```

- └─ APPROVED: their record expires → yours goes ACTIVE
  - └─ REJECTED: nothing moves; your pending record is removed
4. expire → relinquish a record (also cancels a pending transfer)

**The headline:** *You can't accidentally take an asset from another dealer. The API turns the attempt into a request the other dealer must approve — so transfers are consensual and leave an audit trail.*

**Why it matters for how we build:** writes have side effects *beyond the call* — a single “add” can send a real request to another dealership. Integrations must treat writes as deliberate, confirmed actions (never a blind loop), and retry logic must be conflict-aware (re-adding to a PENDING asset is rejected, `400.218`). This is exactly why our tools confirm before writing.

---

## What it does well — and what it deliberately doesn't

### Strengths

- Authoritative, network-wide ownership lookup.
- Policy-enforced, audit-friendly ownership changes and transfers.
- Standard OAuth2 — drops into modern tooling with little glue.
- Fine-grained response shaping (request only the fields you need).

### Intentional limits (set expectations on these)

- **Exact-match search only** — no partial, wildcard, or fuzzy matching.
- **Only four filterable fields** — `dcn`, `serialNumber`, `assetName`, `makeCode`. You **cannot** search by model, year, customer, type, or status.
- **Max two filters, AND only** — no OR, no >2 conditions.
- **No pagination / no “list everything”** — you must search by a known value.
- **Other dealers' DCN numbers are withheld** — you see the customer *name*, not the DCN identifier, for records that aren't yours.

Framing: *“It's a precision instrument, not a reporting warehouse.”* For analytics, we pair it with our own data.

---

## Where we can use it

| Audience      | Opportunity                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------|
| Dealer staff  | Ownership lookup tool; automated onboarding/offboarding; a transfer-request “inbox”                       |
| AI / chat     | M365 Copilot agent — “Who owns serial X?” in natural language ( <i>prototyped</i> )                       |
| Analysts      | Excel <code>=CatLookupSerial()</code> function + batch lookup macro, no code needed ( <i>prototyped</i> ) |
| Data platform | Snowflake UDF to join ownership into the warehouse; Power BI enrichment ( <i>designed</i> )               |
| Governance    | Reconcile internal records vs. the authoritative source; subscription-coverage insight                    |

The pattern: **one small API, many surfaces** — the same four endpoints power chat, spreadsheets, SQL, and dashboards.

## Risks & open questions to work through

- **Production-only, no sandbox.** Every write touches production; mitigated by the isolated test dealer code. *Confirm a safe validation plan before any write integration ships.*
- **Writes have downstream effects** — they can notify another dealer. No automated write loops.
- **Secret management** — where does the client secret live, and who rotates it? Don’t embed it in distributed tools.
- **Entitlement timeline** — credentials start on a test dealer code and are upgraded to real data over ~1–2 weeks. Plan rollout around it.
- **Make-code reference data.** Every add requires a valid `makeCode` / `dealerMakeCode`, and the API validates but does not list them. *We need a source of truth for valid make codes and our dealer make-code mappings before any bulk add/onboarding work — who owns that lookup?*
- **Rate limits** — the API throttles with `429`; bulk tools need backoff. *What are the documented limits for our tier?*

## Key takeaways

- Four endpoints — **search, add/update, expire, transfer** — but it's the authoritative source for asset ownership across the whole dealer network.
- **Search is read-only and network-wide; the other three are governed writes.**
- The smart part is **conflict handling**: claiming an asset another dealer owns doesn't take it — it opens a transfer request they must approve.
- It's **exact-match search on four fields by design** — a precision lookup, not a reporting tool.
- Standard OAuth2 means it **plugs into Copilot, Excel, Snowflake, and Power BI** — we've already prototyped the first three.
- **No sandbox**: everything runs in production against an isolated test dealer code, so our rollout plan must respect that.

---

## Appendix — common error codes

| Code    | Meaning                                           |
|---------|---------------------------------------------------|
| 400.202 | Invalid/missing make code                         |
| 400.204 | Invalid/missing DCN                               |
| 400.210 | DCN not found or not tied to a valid CCID         |
| 400.216 | Invalid model year                                |
| 400.217 | Asset ownership conflict                          |
| 400.218 | A pending transfer already exists                 |
| 403.113 | Credentials not entitled to this dealer/operation |
| 429.x   | Rate limit exceeded                               |

Questions: Asset Management API Support — [AssetManagementApiSupportGroup@cat.com](mailto:AssetManagementApiSupportGroup@cat.com). All actions are viewable in the Customer Admin Tool.